

一、常规部署

1. 环境配置

本机配置

1. 系统：ubuntu22
2. GPU型号：GeForce RTX 3090 24G
3. CUDA版本：11.8
4. Python版本：3.11

1.1 创建虚拟环境

```
conda create -n chatchat python=3.11
conda activate chatchat
conda install ipykernel
ipython kernel install --user --name=chatchat
```

1.2 安装相关依赖

```
# 1. 拉去仓库
git clone https://github.com/chatchat-space/Langchain-Chatchat.git
# 2. 进入目录
cd Langchain-Chatchat
# 3. 安装全部依赖
pip install -r requirements.txt
pip install -r requirements_api.txt
pip install -r requirements_webui.txt
```

2 模型下载

安装 git-lfs

```
# 如果没有安装curl
snap install curl

curl -s https://packagecloud.io/install/repositories/github/git-lfs/script.deb.sh |
sudo bash

sudo apt-get install git-lfs
```

① LLM模型

[Qwen/Qwen1.5-7B-Chat](#)

[Qwen/Qwen1.5-7B](#)

```
git clone https://huggingface.co/Qwen/Qwen1.5-7B
git clone https://huggingface.co/Qwen/Qwen1.5-7B-Chat
```

② Embedding模型

[BAAI/bge-large-zh-v1.5](#)

```
git clone https://huggingface.co/BAAI/bge-large-zh-v1.5
```

③ Reranker模型

[BAAI/bge-reranker-large](#)

```
git clone https://huggingface.co/BAAI/bge-reranker-large
```

```
(chat) root@ubuntu22:~/project/models# tree -L 2
```

```
├── BAAI
│   ├── bge-large-zh-v1.5
│   └── bge-reranker-large
└── Qwen
    ├── Qwen1.5-7B
    └── Qwen1.5-7B-Chat
```

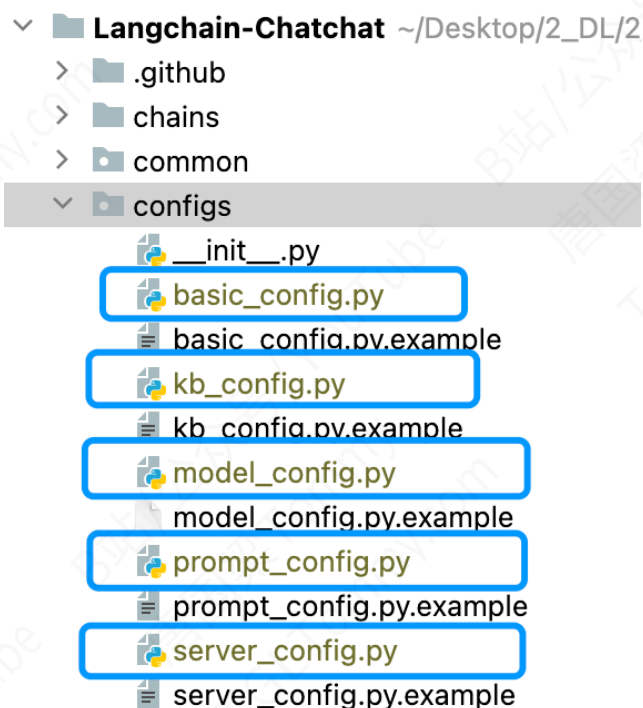
3 参数配置

在开始参数配置之前，先执行以下脚本

```
python copy_config_example.py
```

```
(chat) root@ubuntu22:~/project/Langchain-Chatchat# python copy_config_example.py
(chat) root@ubuntu22:~/project/Langchain-Chatchat#
```

该脚本将会将所有 `config` 目录下的配置文件样例复制一份到 `config` 目录下，方便开发者进行配置。接着，开发者可以根据自己的需求，对配置文件进行修改。



3.1 基础配置项 `basic_config.py`

该配置基负责记录日志的格式和储存路径，通常不需要修改。

3.2 模型配置项 `model_config.py`

本文件包含本地LLM模型、本地Embeddings模型、在线LLM模型API的相关配置。

3.2.1 本地模型路径配置

- 建议将所有下载的模型放到一个统一的目录下，然后将 `MODEL_ROOT_PATH` 指定为该目录，只要模型目录名称符合下列情况之一，即可自动识别加载：

```
MODEL_PATH = {
    "embed_model": {
        "ernie-tiny": "nghuyong/ernie-3.0-nano-zh",
        "ernie-base": "nghuyong/ernie-3.0-base-zh",
        "text2vec-base": "shibing624/text2vec-base-chinese",
        "text2vec": "GanymedeNil/text2vec-large-chinese",
        "text2vec-paraphrase": "shibing624/text2vec-base-chinese-paraphrase",
        "text2vec-sentence": "shibing624/text2vec-base-chinese-sentence",
        "text2vec-multilingual": "shibing624/text2vec-base-multilingual",
        "text2vec-bge-large-chinese": "shibing624/text2vec-bge-large-chinese",
        "m3e-small": "moka-ai/m3e-small",
        "m3e-base": "moka-ai/m3e-base",
        "m3e-large": "moka-ai/m3e-large",

        "bge-small-zh": "BAAI/bge-small-zh",
        "bge-base-zh": "BAAI/bge-base-zh",
        "bge-large-zh": "BAAI/bge-large-zh",
        "bge-large-zh-noinstruct": "BAAI/bge-large-zh-noinstruct",
        "bge-base-zh-v1.5": "BAAI/bge-base-zh-v1.5",
        "bge-large-zh-v1.5": "BAAI/bge-large-zh-v1.5",

        "llm_model": {
            "chatglm2-6b": "THUDM/chatglm2-6b",
            "chatglm2-6b-32k": "THUDM/chatglm2-6b-32k",
            "chatglm3-6b": "THUDM/chatglm3-6b",
            "chatglm3-6b-32k": "THUDM/chatglm3-6b-32k",

            "Orion-14B-Chat": "OrionStarAI/Orion-14B-Chat",
            "Orion-14B-Chat-Plugin": "OrionStarAI/Orion-14B-Chat-Plugin",
            "Orion-14B-LongChat": "OrionStarAI/Orion-14B-LongChat",

            "Llama-2-7b-chat-hf": "meta-llama/Llama-2-7b-chat-hf",
            "Llama-2-13b-chat-hf": "meta-llama/Llama-2-13b-chat-hf",
            "Llama-2-70b-chat-hf": "meta-llama/Llama-2-70b-chat-hf",

            "Qwen-1_8B-Chat": "Qwen/Qwen-1_8B-Chat",
            "Qwen-7B-Chat": "Qwen/Qwen-7B-Chat",
            "Qwen-14B-Chat": "Qwen/Qwen-14B-Chat",
            "Qwen-72B-Chat": "Qwen/Qwen-72B-Chat",
```

3.2.2 在线模型API配置

在 `ONLINE_LLM_MODEL` 已经预先写好了所有支持的在线API服务，通常只需要把申请的API_KEY等填入即可。有些在线API服务需要安装额外的依赖：

```
ONLINE_LLM_MODEL = {  
    "openai-api": {  
        "model_name": "gpt-4",  
        "api_base_url": "https://api.openai.com/v1",  
        "api_key": "",  
        "openai_proxy": "",  
    },  
  
    # 智谱AI API, 具体注册及api key获取请前往 http://open.bigmodel.cn  
    "zhipu-api": {  
        "api_key": "",  
        "version": "glm-4",  
        "provider": "ChatGLMWorker",  
    },  
}
```

3.2.3 HISTORY_LEN

- 历史对话轮数通常不建议设置超过10，因为这可能导致以下问题
 1. 显存占用过高：尤其是部分模型，本身就已经要占用满显存的情况下，保留太多历史，一次传入token太多，可能会爆显存。
 2. 速度处理很慢：因为一次传入了太多token，导致速度很慢。

3.2.4 TEMPERATURE

- 通常不建议设置过高。在Agent对话模式和知识库问答中，我们强烈建议将要其设置成0或者接近于0。

3.2.5 Agent_MODEL = None

- 我们支持用户使用“模型接力赛”的用法，即：选择的大模型仅能调用工具，但是在工具中表现较差，则这个工具作为“模型调用工具”如果用户设置了Agent_MODEL，则在Agent中，使用Agent_MODEL来执行任务，否则，使用LLM_MODEL

3.3 提示词配置项 prompt_config.py

提示词配置分为三个板块，分别对应三种聊天类型。

- llm_chat: 基础的对话提示词，通常来说，直接是用户输入的内容，没有系统提示词。
- knowledge_base_chat: 与知识库对话的提示词，在模板中，我们为开发者设计了一个系统提示词，开发者可以自行更改。
- agent_chat: 与Agent对话的提示词，同样，我们为开发者设计了一个系统提示词，开发者可以自行更改。

prompt模板使用Jinja2语法，简单点就是用双大括号代替f-string的单大括号 请注意，本配置文件支持热加载，修改prompt模板后无需重启服务。

```
PROMPT_TEMPLATES = {  
    "llm_chat": {...},  
  
    "knowledge_base_chat": {...},  
  
    "search_engine_chat": {...},  
  
    "agent_chat": {...}  
}
```

3.4 数据库配置 kb_config.py

3.4.1 分词器配置

请确认本地分词器路径是否已经填写，如：

```
text_splitter_dict = {  
    "ChineseRecursiveTextSplitter": {  
        "source": "huggingface", # 选择tiktoken则使用openai的方法,不填写则默认为字符长度切割方法。  
        "tokenizer_name_or_path": "", # 空格不填则默认使用大模型的分词器。  
    }  
}
```

设置好的分词器需要再 `TEXT_SPLITTER_NAME` 中指定并应用。

在这里，通常使用 `huggingface` 的方法，并且，我们推荐使用大模型自带的分词器来完成任务。

请注意，使用 `gpt2` 分词器将要访问huggingface官网下载权重。

我们还支持使用 `tiktoken` 和传统的 按照长度分词的方式，开发者可以自行配置。

3.4.2 向量数据库配置

`kbs_config` 设置了使用的向量数据库，目前可以选择

- `faiss`: 使用faiss数据库，需要安装faiss-gpu
- `milvus`: 使用milvus数据库，需要安装milvus并进行端口配置
- `pg`: 使用pg数据库，需要配置connection_uri
- `elasticsearch`: 使用ElasticSearch数据库，需要基于docker方式安装和部署

3.5 服务和端口配置项 server_config.py

通常，这个页面并不需要进行大量的修改，仅需确保对应的端口打开，并不互相冲突即可。

如果你是Linux系统推荐设置

```
DEFAULT_BIND_HOST = "0.0.0.0"
```

如果使用联网模型，则需要关注联网模型的端口。

这些模型必须是在model_config.MODEL_PATH或ONLINE_MODEL中正确配置的。

#在启动startup.py时，可用通过 `--model-worker --model-name xxxx` 指定模型，不指定则为LLM_MODEL

4 初始化知识库

当前项目的知识库信息存储在数据库中，在正式运行项目之前请先初始化数据库（我们强烈建议您在执行操作前备份您的知识文件）。

情形一：

如果你是第一次运行本项目，知识库尚未建立，或者之前使用的是低于最新master分支版本的框架，或者配置文件中的知识库类型、嵌入模型发生变化，或者之前的向量库没有开启 `normalize_L2`，需要以下命令初始化或重建知识库：

```
python init_database.py --recreate-vs
```

情形二：

如果你已经有创建过知识库，可以先执行以下命令创建或更新数据库表：

```
python init_database.py --create-tables
```

如果可以正常运行，则无需再重建知识库。

5 一键启动

一键启动脚本 startup.py，一键启动所有 Fastchat 服务、API 服务、WebUI 服务，示例代码：

```
$ python startup.py -a
```

并可使用 `Ctrl + C` 直接关闭所有运行服务。如果一次结束不了，可以多按几次。

可选参数包括 `-a` (或`--all-webui`)，`--all-api`，`--llm-api`，`-c` (或`--controller`)，`--openai-api`，`-m` (或`--model-worker`)，`--api`，`--webui`，其中：

- `--all-webui` 为一键启动 WebUI 所有依赖服务；
- `--all-api` 为一键启动 API 所有依赖服务；
- `--llm-api` 为一键启动 Fastchat 所有依赖的 LLM 服务；
- `--openai-api` 为仅启动 FastChat 的 controller 和 openai-api-server 服务；
- 其他为单独服务启动选项。

```
(chat) root@ubuntu22:~/project/Langchain-Chatchat# python startup.py -a
```

```
=====Langchain-Chatchat Configuration=====
操作系统：Linux-6.2.0-35-generic-x86_64-with-glibc2.35.
python版本：3.11.8 (main, Feb 26 2024, 21:39:34) [GCC 11.2.0]
项目版本：v0.2.10
langchain版本：0.0.354. fastchat版本：0.2.35
```

```
当前使用的分词器：ChineseRecursiveTextSplitter
当前启动的LLM模型：['Qwen1.5-7B-Chat', 'zhipu-api', 'openai-api'] @ cuda
{'device': 'cuda',
 'host': '0.0.0.0',
 'infer_turbo': False,
 'model_path': '/root/project/models/Qwen/Qwen1.5-7B-Chat',
 'model_path_exists': True,
 'port': 20002}
{'api_key': '',
 'device': 'auto',
 'host': '0.0.0.0',
 'infer_turbo': False,
 'online_api': True,
 'port': 21001,
 'provider': 'ChatGLMWorker',
 'version': 'glm-4',
 'worker_class': <class 'server.model_workers.zhipu.ChatGLMWorker'>}
```



```
'port': 21001,
'provider': 'ChatGLMWorker',
'version': 'glm-4',
'worker_class': <class 'server.model_workers.zhipu.ChatGLMWorker'>}
{'api_base_url': 'https://api.openai.com/v1',
'api_key': '',
'device': 'auto',
'host': '0.0.0.0',
'infer_turbo': False,
'model_name': 'gpt-4',
'online_api': True,
'openai_proxy': '',
'port': 20002}
当前Embeddings模型: bge-large-zh-v1.5 @ cuda

服务端运行信息:
OpenAI API Server: http://127.0.0.1:20000/v1
Chatchat API Server: http://127.0.0.1:7861
Chatchat WEBUI Server: http://0.0.0.0:8501
=====Langchain-Chatchat Configuration=====

Collecting usage statistics. To deactivate, set browser.gatherUsageStats to False.

You can now view your Streamlit app in your browser.

URL: http://0.0.0.0:8501
```

6 多卡加载

项目支持多卡加载，需在 startup.py 中的 create_model_worker_app 函数中，修改如下三个参数：

```
gpus=None,
num_gpus= 1,
max_gpu_memory="20GiB"
```

其中，`gpus` 控制使用的显卡的ID，例如 "0,1"；

`num_gpus` 控制使用的卡数；

`max_gpu_memory` 控制每个卡使用的显存容量。

注1：server_config.py的FSCHAT_MODEL_WORKERS字典中也增加了相关配置，如有需要也可通过修改FSCHAT_MODEL_WORKERS字典中对应参数实现多卡加载，且需注意server_config.py的配置会覆盖create_model_worker_app函数的配置。

注2：少数情况下，gpus参数会不生效，此时需要通过设置环境变量CUDA_VISIBLE_DEVICES来指定torch可见的gpu,示例代码：

```
CUDA_VISIBLE_DEVICES=0,1 python startup.py -a
```

二、容器化部署

1. Docker 安装

参考安装链接

```
https://www.runoob.com/docker/ubuntu-docker-install.html
```

2. 安装ElasticSearch和Kibana

2.1 创建网络

```
docker network create elastic
```

2.2 创建Elasticsearch容器并运行

创建容器

```
docker run -id --name es --net elastic -p 9200:9200 -p 9300:9300 \
-e "discovery.type=single-node" \
-e "ES_JAVA_OPTS=-Xms8g -Xmx8g" \
-t docker.elastic.co/elasticsearch/elasticsearch:8.12.2
```

可以查看es日志获取用户名、密码和秘钥

```
docker logs -f es
```

```

✓ Elasticsearch security features have been automatically configured!
✓ Authentication is enabled and cluster connections are encrypted.
  用户名
i Password for the elastic user (reset with `bin/elasticsearch-reset-password -u elastic`):
  qRkqcVe*ufAYUADxcqSf
  密码
i HTTP CA certificate SHA-256 fingerprint:
  eaf4a6cbaa037e06b242c9dc358873d3e53e821f544f382c146032355e09cfb6

i Configure Kibana to use this cluster:
• Run Kibana and click the configuration link in the terminal when Kibana starts.
• Copy the following enrollment token and paste it into Kibana in your browser (valid for the next 30 minutes)
  秘钥 enrollment token
  eyJ2ZXI0i0iI4LjEYlJiLCJhZHII0lsImTcyLjE4LjAuMj05MjAwIl0sImZnciI6ImVhZjRhNmNiYWwMzdLMDZiMjQyYzlkYzM1ODg3M2QzZTUuzZTgyMwY1NDRmMzgyYzE0NjAzMjM1NWUwOWNmYjYiLCJrZXkiOiJYX0Jpam80QnAyRG1IMVNZUk9FRzpx1X1NVTzJDZ1FMwYyYlBLZ3NH
  S0V3In0=

i Configure other nodes to join this cluster:
• Copy the following enrollment token and start new Elasticsearch nodes with `bin/elasticsearch --enrollment-token <token>` (valid for the next 30 minutes):
  eyJ2ZXI0i0iI4LjEYlJiLCJhZHII0lsImTcyLjE4LjAuMj05MjAwIl0sImZnciI6ImVhZjRhNmNiYWwMzdLMDZiMjQyYzlkYzM1ODg3M2QzZTUuzZTgyMwY1NDRmMzgyYzE0NjAzMjM1NWUwOWNmYjYiLCJrZXkiOiJYX0Jpam80QnAyRG1IMVNZUk9FRzpx1X1NVTzJDZ1FMwYyYlBLZ3NH
  S0V3In0=

If you're running in Docker, copy the enrollment token and run:
`docker run -e "ENROLLMENT_TOKEN=<token>" docker.elastic.co/elasticsearch/elasticsearch:8.12.2`

```

重置密码【可选操作】

```
# 进入容器 es
docker exec -it es /bin/bash

# 重置密码
bin/elasticsearch-reset-password -u elastic

# 如下图所示：
# 用户名: elastic
# 密码: CtBLIPPzxZDq1EjKqAQE
```

```
(base) root@ubuntu22:~# docker exec -it es /bin/bash
elasticsearch@abe8610bae46:~$ bin/elasticsearch-reset-password -u elastic
WARNING: Owner of file [/usr/share/elasticsearch/config/users] used to be [root], but now is [elasticsearch]
WARNING: Owner of file [/usr/share/elasticsearch/config/users_roles] used to be [root], but now is [elasticsearch]
This tool will reset the password of the [elastic] user to an autogenerated value.
The password will be printed in the console.
Please confirm that you would like to continue [y/N]y

Password for the [elastic] user successfully reset.
New value: CtBLIPPzxZDq1EjKqAQE
elasticsearch@abe8610bae46:~$
```

2.3 创建Kibana容器并运行

注意：宿主机端口，你可以用自己的端口

```
docker run -id --name kibana --net elastic -p 14765:5601
docker.elastic.co/kibana/kibana:8.12.2
```

2.4 下载IK分词器

从下面的链接打开的页面中，下载IK 8.12.2 版本

```
https://github.com/infinilabs/analysis-ik/releases
```

也可以直接下载

```
wget https://github.com/infinilabs/analysis-ik/releases/download/v8.12.2/elasticsearch-
analysis-ik-8.12.2.zip
```

2.5 将IK分词器压缩包拷贝到Elasticsearch容器内

```
docker cp elasticsearch-analysis-ik-8.12.2.zip es:/usr/share/elasticsearch/plugins
```

2.6 在Elasticsearch容器内，进行解压

```
# ① 进入es容器
docker exec -it es /bin/bash

# ② 进入plugins/
cd plugins

# ③ 创建文件夹 IK
mkdir IK

# ④ 解压
unzip elasticsearch-analysis-ik-8.12.2.zip -d IK/

# ⑤ 删除 elasticsearch-analysis-ik-8.12.2.zip 文件
rm elasticsearch-analysis-ik-8.12.2.zip

# ⑥ 退出容器
exit

# ⑦ 重启es
docker restart es
```

2.7 在浏览器打开kibana窗口

2.7.1 直接用es的密钥登录kibana

请将 2.2步的密钥 输入到这里，并点击 `configure Elastic`：

Configure Elastic to get started

Enrollment token

```
eyJ2ZXliOiI4LjEyLjliLCJhZHliOiMTcyLjE4LjAuMjo5MjAwll0slmZncil6ljdhYTVIZGNIzDc1ZjgxYTZIMDkxN2Q3YmMxY2YyNjZjZDBhNjQ4NDE0MjBjBIYzA3ZmYwY2YxYmI0NWE2Njg5M2MiLCJrZXkiOiJJd0k5aFk0Qjh0ZkpfelV4SXVzVTpTXzZ2SmhNeVFKR25fbm9qMEJhR2xRIn0=
```

Connect to `https://172.18.0.2:9200`

 [Configure manually](#)

[Configure Elastic](#)

2.7.2 生成新的密钥后登录kibana

如果出现下面的页面，则在es容器中输入命令获取tokens：

```
bin/elasticsearch-create-enrollment-token --scope kibana
```

Configure Elastic to get started

Enrollment token

Paste enrollment token from terminal.



Enter an enrollment token.

Where do I find this?

 **Configure manually**

Configure Elastic

Configure Elastic to get started

The enrollment token is automatically generated when you start Elasticsearch for the first time. You might need to scroll back a bit in the terminal to view it.

To generate a new enrollment token, run the following command from the Elasticsearch installation directory:

```
bin/elasticsearch-create-enrollment-token --scope kibana
```



[Learn how to set up Elastic.](#) 

Where do I find this?

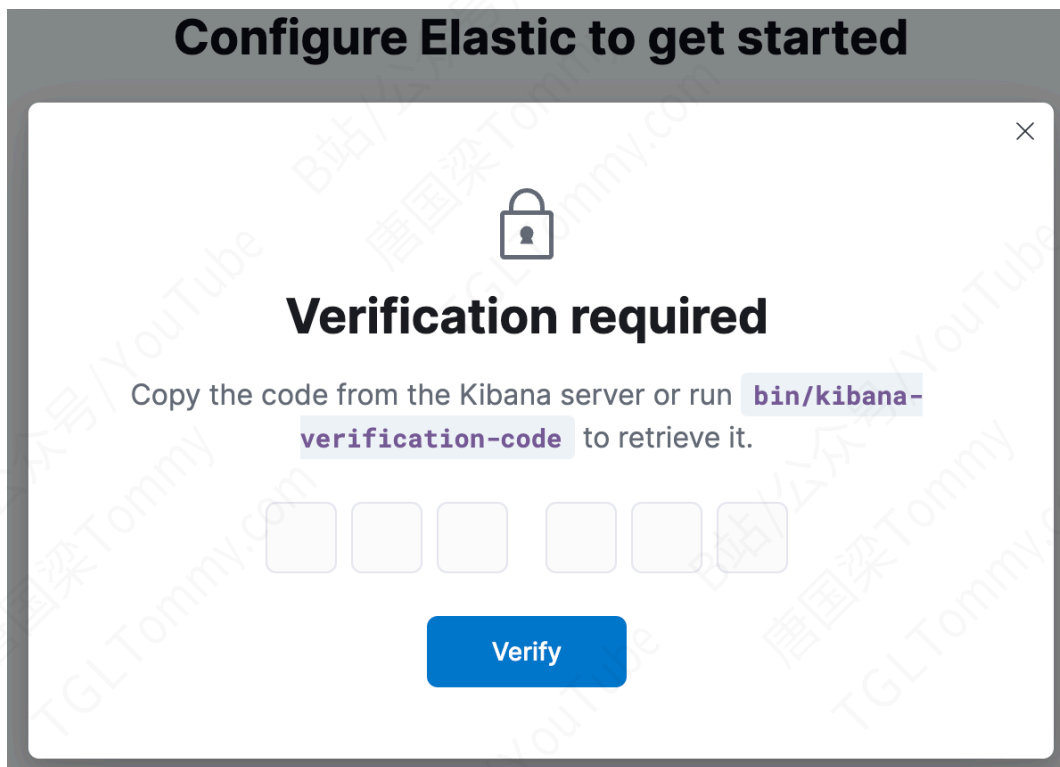
 [Configure manually](#)

Configure Elastic

```
(base) root@ubuntu22:~# docker exec -it es /bin/bash 进入es容器  
elasticsearch@97ba5bc68b6e:~$  
elasticsearch@97ba5bc68b6e:~$ 执行命令  
elasticsearch@97ba5bc68b6e:~$ bin/elasticsearch-create-enrollment-token --scope kibana  
WARNING: Owner of file [/usr/share/elasticsearch/config/users] used to be [root], but now is [elasticsearch]  
WARNING: Owner of file [/usr/share/elasticsearch/config/users_roles] used to be [root], but now is [elasticse  
arch]  
生成的enrollment token  
eyJ2ZXI0iEYlJyLjIiLCJhZHI0LSltMTcyLjE4LjAuMjo5MjAwIl0sImZnciIGlmVHJzRmNmNiYWewMzdldMDZIMjQyYzlkYzM1ODg3M2QzZTUzZTgyMWY1NDhmZGYzYzE0NjAzMjM1NWUwOWNmYjYiLCJrZXkiOiJJZZkjtam80QnAyRG1IMVNZYSlIXzpmQzZWbVNYNVmxdkdUTRI2Wyak13In0=  
elasticsearch@97ba5bc68b6e:~$
```

2.7.3 在kibana容器中生成验证码

如果出现了下面的提示，则需要在kibana容器里进行执行命令：



进入kibana容器，执行下面的命令获取密钥：

```
(base) root@ubuntu22:~# docker exec -it kibana /bin/bash
kibana@336fe8e0fcf1:~$
kibana@336fe8e0fcf1:~$ bin/kibana-verification-code
Kibana is currently running with legacy OpenSSL providers enabled! For details and instructions on how to disable see https://www.elastic.co/guide/en/kibana/8.12/production.html#openssl-legacy-provider
Your verification code is: 001 615
kibana@336fe8e0fcf1:~$
kibana@336fe8e0fcf1:~$
```

刷新一下页面，进入到登录页面。输入用户名和密码，在 2.2步可以查看：

Welcome to Elastic

Username

elastic

Password

8kn8mlPgrsntt5GkdCfa

Log in

最终成功登录 Kibana：

 Find apps, content, and more.

☰

D

Dev Tools

Console

Console

Search Profiler

Grok Debugger

Painless Lab

BETA

History

Settings

Variables

Help

1

Click the Variables button, above, to create your own variables.

2

GET \${exampleVariable1} // _search

3

{

4

"query": {

5

"\${exampleVariable2}": {} // match_all

6

}

7

}

1

3. 在宿主机访问ES服务

3.1 不需要SSL证书验证

```
# es_test.py
```

```
from elasticsearch import Elasticsearch
```

```
# 配置你的Elasticsearch设置
es = Elasticsearch(
    hosts=["https://0.0.0.0:9200"], # IP和端口
    basic_auth=('elastic', 'T8vr7SvyTJOjUZVCXguj'), # 用户名和密码
    verify_certs=False # 不需要SSL验证
)

# 尝试对Elasticsearch实例执行一个简单的请求，获取集群的健康状态
try:
    health = es.cluster.health()
    print("Elasticsearch cluster health:", health)
except Exception as e:
    print("Error connecting to Elasticsearch:", e)
```

```
(chatchat) root@ubuntu22:~/test# python es_test.py
/home/vipuser/miniconda3/envs/chatchat/lib/python3.11/site-packages/elasticsearch/_sync/client/_transport.py:9: SecurityWarning: Connecting to 'https://localhost:9200' using TLS with verify_certs=False is insecure
  _transport = transport_class(
/home/vipuser/miniconda3/envs/chatchat/lib/python3.11/site-packages/urllib3/connectionpool.py:116: RequestWarning: Unverified HTTPS request is being made to host 'localhost'. Adding certificate verification is strongly advised. See: https://urllib3.readthedocs.io/en/latest/advanced-usage.html#tls-warnings
  warnings.warn(
Elasticsearch cluster health: {'cluster_name': 'docker-cluster', 'status': 'green', 'timed_out': False, 'number_of_nodes': 1, 'number_of_data_nodes': 1, 'active_primary_shards': 28, 'active_shards': 28, 'recovery_shards': 0, 'initializing_shards': 0, 'unassigned_shards': 0, 'delayed_unassigned_shards': 0, 'number_of_tasks': 0, 'number_of_in_flight_fetch': 0, 'task_max_waiting_in_queue_millis': 0, 'active_shards_percent': 100.0}
(chatchat) root@ubuntu22:~/test#
```